research/understanding-and-hardening-linux-containers/).

In most cases, a pod should contain only a single main container and that container should run only a single process. If the process (or node) dies, Kubernetes will restart the pod automatically, possibly on a different node. There are two general exceptions to the one-container-per-pod rule:

- An *init container* runs before any other containers in the pod and can be used to perform initialization tasks, such as waiting for other services to become available. The main container in a pod will not be started until all init containers have completed.
- A *sidecar container* runs alongside the main container and provides additional services. For example, a sidecar container might implement a reverse proxy for a web server running in the main container, or it might periodically update data files on a filesystem shared with the main container.

For the most part, you don’t need to worry about these different kinds of containers in this chapter and can stick to the one-container-per-pod rule. You’ll see an example of a sidecar container when you learn about the Linkerd service mesh in section 10.3.2.

A Kubernetes cluster can be highly dynamic with pods being created and destroyed or moved from one node to another to achieve performance and availability goals. This makes it challenging for a container running in one pod to call an API running in another pod, because the IP address may change